



UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CAMPUS PETROLINA

Éricles Barros de Sá

ROBÓTICA I

Planejamento Híbrido de Trajetórias: Fusão A* Otimizado e DWA

Petrolina, PE

2026

Éricles Barros de Sá

ROBÓTICA I

Planejamento Híbrido de Trajetórias: Fusão A* Otimizado e DWA

Trabalho requerido pelo professor Juracy Emanuel França, como parte das exigências de avaliação da disciplina optativa de Robótica 1, do curso de Engenharia da Computação, da Universidade Federal do Vale do São Francisco.

Petrolina, PE

2026

SUMÁRIO

1. Introdução.....	4
2. O Algoritmo Híbrido: A* Otimizado e DWA.....	5
2.1. A* Otimizado com Vizinhança Expandida (16 Direções).....	5
2.2. Extração de Pontos-Chave por Poda Geométrica.....	6
2.3. DWA como Controlador Subordinado.....	6
3. Metodologia e Implementação.....	7
3.1. Sensores, Percepção e Planta Robótica.....	7
3.1.1. O Mapa de Grade de Ocupação (Occupancy Grid Map).....	8
3.2. Infraestrutura de Comunicação e Sincronismo Absoluto.....	8
3.3. Arquitetura de Decisão Hierárquica em Prioridades.....	9
3.4. Bloco 1.5: Centralização Proporcional em Corredores.....	10
3.5. Correção do Fator Angular Fixo na Prioridade 2.....	10
3.6. Salvaguarda de Emergência.....	11
4. Resultados e Discussão.....	11
5. Anexo: Código Fonte (Python).....	12
6. REFERÊNCIAS.....	30

1. Introdução

Este relatório descreve a implementação, os testes e os resultados da aplicação de um algoritmo híbrido de navegação autônoma, que combina um planejador global baseado no A* Otimizado com um controlador local reativo do tipo DWA (Dynamic Window Approach). O objetivo do trabalho consistiu em fazer com que um modelo 3D do robô iRobot Create 2, inserido no simulador CoppeliaSim, fosse capaz de perceber o ambiente por meio de uma câmera de visão, planejar uma rota global até uma coordenada alvo (Target) e, ao mesmo tempo, reagir em tempo real a obstáculos dinâmicos não previstos no mapa, como pedestres.

A concepção deste algoritmo se apoia na estrutura proposta por Li, Hu, Wang e Du (2020), publicada nos anais da 2nd International Conference on Artificial Intelligence and Advanced Manufacture (AIAM), que defende a fusão de um planejador global de trajetórias com um controlador dinâmico local como forma de eliminar as fraquezas individuais de cada método isolado. Diferente do trabalho anterior sobre o Método da Janela Dinâmica, no qual o DWA atuava sozinho do início ao fim da navegação, este projeto evolui a arquitetura para um modelo verdadeiramente hierárquico, no qual o A* Otimizado define o caminho estrutural (a espinha dorsal da rota) e o DWA é acionado apenas de forma subordinada e reativa. Assim como no trabalho anterior, a lógica de controle foi desenvolvida externamente em linguagem Python, utilizando a biblioteca ZeroMQ Remote API, o que garante comunicação em tempo real entre a IDE (VS Code) e a engine de física do CoppeliaSim em modo síncrono.

2. O Algoritmo Híbrido: A* Otimizado e DWA

O modelo de fusão híbrida parte do princípio de que o planejamento de trajetórias em robótica autônoma pode ser subdividido em duas camadas hierárquicas complementares:

- Camada Global (Planejador): calcula uma rota preliminar sobre um mapa de ocupação conhecido, priorizando eficiência algorítmica e caminhos mais curtos. Utiliza o algoritmo A*.
- Camada Local (Controlador): responsável pela reatividade e pelo desvio de obstáculos dinâmicos e imprevisos, atuando em tempo real sobre o espaço de velocidades do robô. Utiliza o algoritmo DWA.

Essa divisão resolve as fraquezas que cada método apresenta isoladamente. O A* convencional, restrito a uma vizinhança de 8 direções (múltiplos de 45°), gera trajetórias serrilhadas, com curvas agudas incompatíveis com a cinemática real de um robô diferencial. Já o DWA puro, sendo estritamente reativo e sem visão global do mapa, é vulnerável a mínimos locais em cenários de corredores ou obstáculos em formato de "U", podendo travar o robô mesmo com o alvo visível.

2.1. A* Otimizado com Vizinhança Expandida (16 Direções)

Para reduzir a severidade angular do A* tradicional, o planejador global foi implementado com um modelo de movimento expandido, que soma às 8 direções clássicas mais 8 direções adicionais de salto duplo (passos do tipo $(\pm 2, \pm 1)$ e $(\pm 1, \pm 2)$), totalizando 16 direções de busca. Essa técnica é inspirada na busca de cinco comprimentos de lado (five-side-length search) descrita por Li et al. (2020), que expande a vizinhança convencional de 3x3 para uma varredura equivalente a uma matriz 5x5, suavizando as transições angulares da rota.

$$f(n) = g(n) + h(n)$$

A função de custo do A* segue a formulação clássica, em que $g(n)$ representa o custo real acumulado do caminho percorrido até o nó n , e $h(n)$ é a heurística de distância até o destino. Neste projeto, no entanto, $h(n)$ não é fixa: ela é ponderada dinamicamente por um fator de densidade de obstáculos calculado ao redor de cada nó candidato, dentro de um raio de varredura local.

$$h_{\text{adaptativa}}(n) = \text{peso} \times \text{dist_euclidiana}(n, \text{alvo}), \text{ com peso} \in [0,5 ; 1,5]$$

Quando o robô se encontra em uma região aberta, com baixa densidade de obstáculos ao redor, o peso da heurística se aproxima do limite superior (1,5),

acelerando a convergência do algoritmo em linha reta na direção do destino. Em regiões de alta densidade de obstáculos, o peso decresce em direção a 0,5, fazendo com que o planejador priorize o custo real acumulado $g(n)$ e explore o espaço com mais cautela, evitando armadilhas de becos e reduzindo buscas desnecessárias em nós colaterais.

2.2. Extração de Pontos-Chave por Poda Geométrica

A rota bruta produzida pela busca A* contém um número excessivo de nós intermediários, muitos deles redundantes por estarem alinhados em segmentos retos. Para simplificar essa rota em um conjunto mínimo de submetas, foi implementado um mecanismo de poda em duas etapas:

- Filtragem por variação angular: percorre-se a rota bruta comparando o ângulo de cada segmento consecutivo com o segmento anterior; pontos cuja mudança de direção excede 15° são retidos como candidatos a nós críticos, e os demais são descartados.
- Poda por linha de visada (Line-of-Sight): a partir de cada nó crítico retido, verifica-se, por amostragem ao longo do segmento reto, se existe conexão direta e livre de colisão até o nó crítico mais distante possível na lista; se houver linha de visada, os nós intermediários são eliminados e a rota é encurtada; caso contrário, o algoritmo retrocede um nó e retenta.

O resultado desse processo é uma sequência enxuta de pontos-chave (key_x , key_y), compreendendo exclusivamente os vértices de transição direcional realmente necessários para descrever a rota, o que reduz o estresse mecânico do robô ao eliminar acelerações e frenagens bruscas decorrentes de curvas desnecessárias.

2.3. DWA como Controlador Subordinado

Diferentemente da implementação anterior, em que o DWA governava toda a navegação, neste algoritmo híbrido o DWA atua apenas como uma camada de decisão de último recurso, acionada exclusivamente quando o robô se encontra em um trecho de ambiente aberto, sem correspondência clara com a rota global mapeada pelo A*. Nos demais casos, o controle é assumido por blocos de decisão hierárquicos e mutuamente exclusivos, descritos na Seção 3.3.

Quando efetivamente acionado, o DWA segue a formulação clássica: gera um espaço de velocidades possíveis dentro da janela dinâmica delimitada pelas acelerações máximas do robô, projeta trajetórias em arco de curto horizonte para cada par (v, ω) candidato e escolhe aquele que maximiza a função objetivo:

$$G(v, \omega) = \alpha \cdot \text{Heading}(v, \omega) + \beta \cdot \text{Clearance}(v, \omega) + \gamma \cdot \text{Velocity}(v, \omega)$$

Os pesos utilizados nesta implementação foram $\alpha = 4,0$, $\beta = 0,8$ e $\gamma = 2,0$, priorizando fortemente o alinhamento angular em relação à submeta ativa (Heading), com peso intermediário para velocidade e peso reduzido para distanciamento de obstáculos, uma vez que a segurança contra colisões já é majoritariamente resolvida pelas camadas de prioridade superiores descritas na Seção 3.3, e não pelo próprio termo de Clearance.

3. Metodologia e Implementação

3.1. Sensores, Percepção e Planta Robótica

A base robótica utilizada foi o modelo tridimensional do iRobot Create 2, mantendo os mesmos 5 sensores de proximidade frontais (Esquerda, Diagonal Esquerda, Meio, Diagonal Direita e Direita) do trabalho anterior, com alcance de detecção ajustado para 0,5 metros. Os parâmetros físicos do fabricante também foram mantidos: raio das rodas motrizes $R = 0,036$ m e distância entre eixos $L = 0,235$ m.

A principal evolução em relação ao trabalho anterior está na percepção do ambiente para o planejamento global: em vez de depender apenas dos sensores de proximidade, este algoritmo utiliza um sensor de visão (câmera ortográfica) posicionado sobre o cenário, cuja imagem é convertida em um Mapa de Ocupação Binário (Occupancy Grid Map - OGM). O processamento segue as seguintes etapas:

- Captura da imagem bruta via `sim.getVisionSensorImg` e conversão para escala de cinza através da média dos canais RGB.
- Binarização por limiar (`threshold = 245`), separando pixels de obstáculo dos pixels de piso livre.
- Conversão de cada pixel obstruído em coordenadas cartesianas do mundo real, considerando a posição da câmera e o tamanho ortográfico do sensor.
- Descarte de pontos situados a menos de 0,35 m da posição inicial do robô, evitando que o próprio chassi do veículo seja interpretado como obstáculo.

O mapa de pontos (ox, oy) resultante alimenta diretamente o planejador A*, que infla os obstáculos com o raio de segurança do robô (0,15 m) antes de iniciar a busca.

3.1.1. O Mapa de Grade de Ocupação (Occupancy Grid Map)

A criação do Mapa de Grade de Ocupação (Occupancy Grid Map - OGM) da cena no CoppeliaSim foi realizada de forma simplificada: obtém-se uma imagem binária da cena a partir de um sensor de visão posicionado acima da superfície do ambiente e, em seguida, analisa-se, célula a célula, se cada região da imagem obtida corresponde a um espaço livre ou a um espaço ocupado por obstáculo. Essa abordagem dispensa a necessidade de sensores de varredura a laser (LiDAR) ou de fusão sensorial complexa para a etapa de mapeamento estático, uma vez que toda a informação estrutural do ambiente é extraída diretamente da imagem capturada pela câmera ortográfica.

Neste projeto, a imagem bruta da câmera é convertida para escala de cinza e binarizada por um limiar fixo (threshold = 245): pixels acima desse valor são interpretados como piso livre, e os demais como células ocupadas. Cada pixel classificado como ocupado é então reprojetoado para coordenadas cartesianas do mundo real, compondo a nuvem de pontos (ox, oy) que representa o OGM da cena e que serve de entrada direta para a construção da grade de obstáculos do planejador A*.

O conceito de mapa de grade de ocupação utilizado nesta implementação tem como referência base o trabalho seminal de Moravec e Elfes (1984), que introduziu a representação probabilística do ambiente em células de ocupação a partir de leituras sonoras de longo alcance, estabelecendo as bases metodológicas para a construção de mapas de grade amplamente empregados até hoje em robótica móvel.

3.2. Infraestrutura de Comunicação e Sincronismo Absoluto

Mantendo a solução validada no trabalho anterior, este projeto também utiliza a ZeroMQ Remote API (coppeliasim_zmqremoteapi_client), com o ecossistema de simulação configurado em Modo Síncrono Estrito por meio do comando client.setStepping(True). Sob essa topologia, o motor de física do CoppeliaSim congela o ambiente a cada passo discreto, aguardando que o script Python

processe a visão computacional, execute o planejamento e/ou a decisão reativa, e despache os comandos de velocidade através de `client.step()`, eliminando o jitter e garantindo controle determinístico.

3.3. Arquitetura de Decisão Hierárquica em Prioridades

O núcleo deste algoritmo é uma máquina de decisão estruturada em blocos de prioridade mutuamente exclusivos, avaliados sequencialmente a cada ciclo de controle. Essa arquitetura resolve, de forma explícita, o requisito de que o DWA deve permanecer estritamente subordinado ao planejamento A*, sendo acionado apenas quando nenhuma outra condição de prioridade superior se aplica.

Prioridade	Condição de Disparo	Comportamento
1 Desvio Frontal Crítico	Sensor central detecta obstáculo a menos de 0,25 m (ex.: pedestre)	Reduz drasticamente a velocidade linear e gira em direção ao lado de maior espaço livre, com reação imediata e incondicional
1.5 Centralização Lateral	Diferença relevante entre distâncias laterais esquerda/direita em corredor estreito	Bloco proporcional que centraliza o robô entre as paredes, com filtro passa-baixa para suavizar a variação angular
2 Confiança no A*	Obstáculo detectado corresponde a uma célula já mapeada no OGM global	Controlador proporcional simples: acelera e corrige o erro de heading em relação à submeta ativa, sem replanejamento
3 Passagem Estreita	Obstáculos simultâneos à esquerda e à direita, sem obstrução frontal	Alinhamento suave e conservador para atravessar o corredor sem oscilação
4 DWA Reativo	Nenhuma das condições anteriores se aplica (ambiente aberto ou obstáculo não mapeado)	Busca completa no espaço de velocidades, avaliando Heading, Clearance e Velocity

Essa hierarquia garante que o DWA (Prioridade 4) só assuma o controle quando o robô está em uma região aberta do mapa ou diante de um obstáculo dinâmico não previsto na fase de planejamento global, validando na prática o requisito de que a Janela Dinâmica deve ser subordinada ao A*, e não a lógica dominante da navegação.

3.4. Bloco 1.5: Centralização Proporcional em Corredores

Durante os testes, identificou-se que o robô apresentava um leve desalinhamento lateral ao atravessar corredores estreitos, aproximando-se demais de uma das paredes mesmo sem risco imediato de colisão frontal. Para corrigir esse comportamento, foi adicionado o bloco de PRIORIDADE 1.5, um controlador proporcional dedicado exclusivamente à centralização lateral do robô.

O bloco compara a menor distância lateral esquerda (d_{esq}) com a menor distância lateral direita (d_{dir}) e calcula um erro de centralização:

$$\text{erro_centro} = d_{\text{dir}} - d_{\text{esq}}$$

Esse erro é convertido em uma velocidade angular alvo por meio de um ganho proporcional, limitada a $\pm 0,25$ rad/s, e suavizada por um filtro passa-baixa (limitando a variação máxima de ω a 0,8 rad/s por passo de simulação), evitando movimentos bruscos de correção. O bloco só é ativado quando a diferença lateral ultrapassa 0,12 m e é automaticamente desligado quando a diferença cai abaixo de 0,08 m ou após 30 ciclos consecutivos, prevenindo oscilações de entrada e saída do modo de centralização.

3.5. Correção do Fator Angular Fixo na Prioridade 2

Na primeira versão do bloco de PRIORIDADE 2 (Confiança no A*), a velocidade angular de correção era calculada com um ganho fixo aplicado apenas quando o erro de heading excedia 0,5 radianos, resultando em curvas abruptas e descontínuas ao sair da rota reta. A correção aplicada consistiu em substituir esse valor fixo por uma lei proporcional contínua:

$$w_{\text{alvo}} = \text{limitar}(-0,8 ; 0,8 ; 1,5 \times \text{erro_angular})$$

Combinada a um filtro passa-baixa de suavização ($\max_{\text{dw}} = 1,2 \times dt$), essa mudança eliminou os saltos angulares bruscos e tornou a transição entre o alinhamento reto e a correção de curva mais suave e previsível, mantendo a rota fiel aos pontos-chave gerados pelo A*.

3.6. Salvaguarda de Emergência

Como camada final de segurança, independente da hierarquia de prioridades, o algoritmo mantém uma verificação de distância crítica absoluta: caso qualquer um dos 5 sensores detecte um obstáculo a menos de 0,10 m, o robô é forçado a recuar levemente ($v = -0,02$ m/s) e a girar em direção ao lado de maior espaço livre, sobrepondo-se a qualquer decisão da Prioridade 4 (DWA).

4. Resultados e Discussão

A validação experimental do algoritmo híbrido confirmou que a arquitetura hierárquica de prioridades atingiu o objetivo de manter o DWA estritamente subordinado ao planejamento global do A^* . Nos trechos em que o mapa de ocupação corretamente antecipava os obstáculos presentes na cena, o robô permaneceu majoritariamente sob controle da Prioridade 2, seguindo a rota planejada com correções suaves de heading, sem necessidade de recorrer à busca completa do espaço de velocidades.

O comportamento de centralização (Prioridade 1.5) mostrou-se eficaz para eliminar o desalinhamento lateral observado nos primeiros testes em corredores estreitos, mantendo o robô equidistante das paredes sem introduzir oscilações perceptíveis, graças ao filtro passa-baixa aplicado sobre a variação angular.

A ativação da Prioridade 1 (Desvio Frontal Crítico) foi testada especificamente em cenários de obstáculo dinâmico do tipo pedestre, cruzando a trajetória planejada do robô. Em todos os ensaios, o robô interrompeu antecipadamente seu avanço linear, executou uma manobra evasiva em direção ao lado de maior espaço livre e, assim que o obstáculo deixou de ser detectado pelos sensores frontais, retomou automaticamente o controle hierárquico normal, sem necessidade de replanejamento do A^* .

A camada de DWA puro (Prioridade 4) foi solicitada nos trechos em que o robô se encontrava em áreas abertas do mapa, sem correspondência com obstáculos previamente mapeados, confirmando que sua atuação permaneceu restrita a um papel reativo e complementar, e não à condução primária da navegação, validando o princípio de subordinação hierárquica proposto na arquitetura de fusão de Li et al. (2020).

5. Anexo: Código Fonte (Python)

Abaixo encontra-se a implementação completa utilizada na validação dos testes:

```
# Algoritmo 2 - Éricles
# Híbrido A* e DWA

import math
import time
import numpy as np
import matplotlib.pyplot as plt
from coppeliasim_zmqremoteapi_client import RemoteAPIClient

# Planejador Global A* Otimizado (Li et al., 2020)
class AStarPlanner:
    # Inicializa parâmetros do planejador global
    def __init__(self, ox, oy, resolution, rr):
        self.resolution = resolution
        self.rr = rr
        self.min_x, self.min_y = 0, 0
        self.max_x, self.max_y = 0, 0
        self.obstacle_map = None
        self.x_width, self.y_width = 0, 0
        self.motion = self.get_motion_model()
        self.calc_obstacle_map(ox, oy)

    # Estrutura de dados para os nós da grade
    class Node:
        def __init__(self, x, y, cost, parent_index):
            self.x = x
            self.y = y
            self.cost = cost
            self.parent_index = parent_index

    # Modelo de movimento expandido para 16 direções (matriz 5x5)
    @staticmethod
    def get_motion_model():
        return [
            [1, 0, 1], [0, 1, 1], [-1, 0, 1], [0, -1, 1],
            [1, 1, math.sqrt(2)], [-1, 1, math.sqrt(2)],
```

```

        [1, -1, math.sqrt(2)], [-1, -1, math.sqrt(2)],
        [2, 1, math.sqrt(5)], [1, 2, math.sqrt(5)],
        [-1, 2, math.sqrt(5)], [-2, 1, math.sqrt(5)],
        [-2, -1, math.sqrt(5)], [-1, -2, math.sqrt(5)],
        [1, -2, math.sqrt(5)], [2, -1, math.sqrt(5)]
    ]

# Calcula a densidade local de obstáculos ao redor do nó
def calculate_obstacle_density(self, node, radius=3):
    count, total = 0, 0
    for dx in range(-radius, radius + 1):
        for dy in range(-radius, radius + 1):
            nx, ny = node.x + dx, node.y + dy
            if 0 <= nx < self.x_width and 0 <= ny < self.y_width:
                total += 1
                if self.obstacle_map[nx][ny]:
                    count += 1
    return count / total if total > 0 else 0

# Heurística adaptativa baseada na densidade de barreiras
def calc_heuristic_adaptive(self, n1, n2):
    base = math.hypot(n1.x - n2.x, n1.y - n2.y)
    density = self.calculate_obstacle_density(n1)
    weight = max(0.5, min(1.5, 1.5 - density))
    return weight * base

# Algoritmo de poda geométrica para extração de submetas críticas
def extract_critical_points_los(self, rx, ry):
    if len(rx) <= 2:
        return rx, ry

    key_x, key_y = [rx[0]], [ry[0]]
    dx1, dy1 = rx[1] - rx[0], ry[1] - ry[0]
    angulo_anterior = math.atan2(dy1, dx1)

    # Filtra pontos por mudança brusca de direção (>15 graus)
    for i in range(2, len(rx)):
        dx, dy = rx[i] - rx[i-1], ry[i] - ry[i-1]

```

```

        angulo_atual = math.atan2(dy, dx)
        diff = abs(math.atan2(math.sin(angulo_atual - angulo_anterior),
                                math.cos(angulo_atual - angulo_anterior)))

        if diff > math.radians(15):
            key_x.append(rx[i-1])
            key_y.append(ry[i-1])
            angulo_anterior = angulo_atual

    if (key_x[-1], key_y[-1]) != (rx[-1], ry[-1]):
        key_x.append(rx[-1])
        key_y.append(ry[-1])

    # Poda por linha de visada direta (Line-of-Sight)
    final_x, final_y = [key_x[0]], [key_y[0]]
    curr = 0

    while curr < len(key_x) - 1:
        look_ahead = len(key_x) - 1
        found = False
        while look_ahead > curr + 1:
            if self.has_line_of_sight_simple(key_x[curr], key_y[curr],
key_x[look_ahead], key_y[look_ahead]):
                final_x.append(key_x[look_ahead])
                final_y.append(key_y[look_ahead])
                curr = look_ahead
                found = True
                break
            look_ahead -= 1

        if not found:
            curr += 1
            if curr < len(key_x):
                final_x.append(key_x[curr])
                final_y.append(key_y[curr])

    if (final_x[-1], final_y[-1]) != (rx[-1], ry[-1]):
        final_x.append(rx[-1])

```

```

        final_y.append(ry[-1])

    return final_x, final_y

# Amostragem linear para verificação de colisão em linha reta
def has_line_of_sight_simple(self, x1, y1, x2, y2, step_size=0.15):
    dist = math.hypot(x2 - x1, y2 - y1)
    steps = max(int(dist / step_size), 2)
    for i in range(steps + 1):
        t = i / steps
        cx, cy = x1 + t * (x2 - x1), y1 + t * (y2 - y1)
        node = self.Node(self.calc_xy_index(cx, self.min_x),
self.calc_xy_index(cy, self.min_y), 0.0, -1)
        if not self.verify_node(node):
            return False
    return True

# Laço principal de busca do A* global
def planning(self, sx, sy, gx, gy):
    start_node = self.Node(self.calc_xy_index(sx, self.min_x),
self.calc_xy_index(sy, self.min_y), 0.0, -1)
    goal_node = self.Node(self.calc_xy_index(gx, self.min_x),
self.calc_xy_index(gy, self.min_y), 0.0, -1)

    if not self.verify_node(start_node) or not
self.verify_node(goal_node):
        print("[ERRO FATAL] Ponto de partida ou alvo inválido!")
        return [], []

    open_set = {self.calc_grid_index(start_node): start_node}
    closed_set = {}

    while open_set:
        c_id = min(open_set, key=lambda o: open_set[o].cost +
self.calc_heuristic_adaptive(goal_node, open_set[o]))
        current = open_set[c_id]

        if current.x == goal_node.x and current.y == goal_node.y:

```

```

        goal_node.parent_index = current.parent_index
        goal_node.cost = current.cost
        break

    del open_set[c_id]
    closed_set[c_id] = current

    for motion in self.motion:
        node = self.Node(current.x + motion[0], current.y + motion[1],
current.cost + motion[2], c_id)
        n_id = self.calc_grid_index(node)

        if not self.verify_node(node) or n_id in closed_set:
            continue

        if n_id not in open_set or open_set[n_id].cost > node.cost:
            open_set[n_id] = node

    if not open_set:
        print("[ERRO] Caminho global não encontrado!")
        return [], []

    return self.calc_final_path(goal_node, closed_set)

# Reconstrói a lista de coordenadas finais de trás para frente
def calc_final_path(self, goal_node, closed_set):
    rx, ry = [self.calc_grid_position(goal_node.x, self.min_x)],
[self.calc_grid_position(goal_node.y, self.min_y)]
    parent_index = goal_node.parent_index
    while parent_index != -1:
        n = closed_set[parent_index]
        rx.append(self.calc_grid_position(n.x, self.min_x))
        ry.append(self.calc_grid_position(n.y, self.min_y))
        parent_index = n.parent_index
    return rx, ry

# Converte índice da matriz para metros no mundo real
def calc_grid_position(self, index, min_position):

```



```

        return index * self.resolution + min_position

# Converte metros do mundo real para índice discreto da matriz
def calc_xy_index(self, position, min_pos):
    return round((position - min_pos) / self.resolution)

# Gera chave linear única para armazenamento em dicionários
def calc_grid_index(self, node):
    return (node.y - self.min_y) * self.x_width + (node.x - self.min_x)

# Valida se o nó está contido no espaço navegável da grade
def verify_node(self, node):
    px, py = self.calc_grid_position(node.x, self.min_x),
self.calc_grid_position(node.y, self.min_y)
    if px < self.min_x or py < self.min_y or px >= self.max_x or py >=
self.max_y: return False
    if node.x < 0 or node.x >= self.x_width or node.y < 0 or node.y >=
self.y_width: return False
    return not self.obstacle_map[node.x][node.y]

# Constrói a matriz booleana inflando barreiras com o raio do robô
def calc_obstacle_map(self, ox, oy):
    if not ox or not oy: return
    self.min_x, self.min_y = min(ox), min(oy)
    self.max_x, self.max_y = max(ox), max(oy)
    self.x_width = round((self.max_x - self.min_x) / self.resolution) + 1
    self.y_width = round((self.max_y - self.min_y) / self.resolution) + 1
    self.obstacle_map = [[False] * self.y_width for _ in
range(self.x_width)]

    for ix in range(self.x_width):
        x = self.calc_grid_position(ix, self.min_x)
        for iy in range(self.y_width):
            y = self.calc_grid_position(iy, self.min_y)
            for iox, ioy in zip(ox, oy):
                if math.hypot(iox - x, ioy - y) <= self.rr:
                    self.obstacle_map[ix][iy] = True
                    break

```

```

# Processa os dados da câmera ortográfica e gera nuvem de pontos
def get_occupancy_grid_from_vision(sim, sensor_handle, sx, sy, threshold=245):
    img_bytes, res = sim.getVisionSensorImg(sensor_handle)
    res_x, res_y = res[0], res[1]

    img_np = np.frombuffer(img_bytes, dtype=np.uint8).reshape(res_y, res_x, 3)
    gray = np.mean(img_np, axis=2)
    binary_map = gray > threshold

    # Exibe as imagens brutas e binarizadas do mapa
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.title("Visão Bruta da Câmera")
    plt.imshow(gray, cmap='gray', origin='lower')
    plt.subplot(1, 2, 2)
    plt.title("Mapa de Ocupação OGM")
    plt.imshow(binary_map, cmap='gray', origin='lower')
    plt.show()

    ortho_size = sim.getObjectFloatParam(sensor_handle,
sim.visionfloatparam_ortho_size)
    cam_pos = sim.getObjectPosition(sensor_handle, -1)
    pixel_size_x, pixel_size_y = ortho_size / res_x, ortho_size / res_y
    ox, oy = [], []

    # Transforma índices de pixels em coordenadas cartesianas reais
    for y in range(res_y):
        for x in range(res_x):
            if binary_map[y, x]:
                world_x = cam_pos[0] + (ortho_size / 2) - (x * pixel_size_x)
                world_y = cam_pos[1] + (ortho_size / 2) - (y * pixel_size_y)
                if math.hypot(world_x - sx, world_y - sy) > 0.35:
                    ox.append(world_x)
                    oy.append(world_y)

    return ox, oy, min(pixel_size_x, pixel_size_y)

```

```

# Verifica ocupação estática de uma célula no mapa global OGM
def is_obstacle_in_ogm(x, y, a_star, grid_size):
    try:
        ix = round((x - a_star.min_x) / grid_size)
        iy = round((y - a_star.min_y) / grid_size)
        if 0 <= ix < a_star.x_width and 0 <= iy < a_star.y_width:
            return a_star.obstacle_map[ix][iy]
    except:
        pass
    return False

# Mapeia os ângulos construtivos de montagem do array de sensores
def get_sensor_angle(sensor_index):
    return [-math.pi/3, -math.pi/6, 0, math.pi/6, math.pi/3][sensor_index]

# Inicialização e loop principal de controle híbrido
def main():
    print("=" * 60)
    print("  ALGORITMO HÍBRIDO - A* OTIMIZADO + DWA (LÓGICA ORIGINAL SUAVIZADA)")
    print("=" * 60)

    # Instancia o cliente remoto e handles do CoppeliaSim
    client = RemoteAPIClient()
    sim = client.require('sim')

    motorEsquerdo = sim.getObject("/Cuboid/MOTOR_ESQUERDO")
    motorDireito = sim.getObject("/Cuboid/MOTOR_DIREITO")
    robotBase = sim.getObject("/Cuboid")
    vision_sensor = sim.getObject("/Vision_sensor")
    goalHandle = sim.getObject("/Target")

    sensores = [
        sim.getObject("/Cuboid/SENSOR_ESQUERDO"),
        sim.getObject("/Cuboid/SENSOR_DIAG_ESQUERDO"),
        sim.getObject("/Cuboid/SENSOR_MEIO"),
        sim.getObject("/Cuboid/SENSOR_DIAG_DIREITO"),
        sim.getObject("/Cuboid/SENSOR_DIREITO")
    ]

```

```

]

# Inicia ambiente síncrono de simulação
client.setStepping(True)
sim.startSimulation()

for _ in range(10):
    client.step()
    time.sleep(0.05)

# Coleta telemetria e posições iniciais
start_pos = sim.getObjectPosition(robotBase, -1)
final_goal_pos = sim.getObjectPosition(goalHandle, -1)
sx, sy = start_pos[0], start_pos[1]
gx, gy = final_goal_pos[0], final_goal_pos[1]

# Executa mapeamento por visão computacional
ox, oy, sensor_res = get_occupancy_grid_from_vision(sim, vision_sensor,
sx, sy)
grid_size = max(sensor_res * 2, 0.15)
robot_radius = 0.15

# Planeja rota e extrai nós críticos
a_star = AStarPlanner(ox, oy, grid_size, robot_radius)
rx, ry = a_star.planning(sx, sy, gx, gy)

if not rx:
    sim.stopSimulation()
    return

rx, ry = rx[::-1], ry[::-1]
key_x, key_y = a_star.extract_critical_points_los(rx, ry)

# Plota o mapa de rotas estáticas calculadas
plt.figure(figsize=(12, 10))
plt.plot(ox, oy, ".k", markersize=2, alpha=0.5)
plt.plot(sx, sy, "og", markersize=14, zorder=5)
plt.plot(gx, gy, "Xb", markersize=16, zorder=5)

```

```

plt.plot(rx, ry, color="lightcoral", linestyle="--", linewidth=2,
alpha=0.7)
plt.plot(key_x, key_y, color="magenta", linewidth=4)
if len(key_x) > 2: plt.scatter(key_x[1:-1], key_y[1:-1],
color='saddlebrown', s=150, zorder=5)
plt.scatter(key_x[0], key_y[0], color='green', s=250, zorder=6,
marker='o')
plt.scatter(key_x[-1], key_y[-1], color='blue', s=250, zorder=6,
marker='x')
plt.grid(True, linestyle=":", alpha=0.3)
plt.axis("equal")
plt.gca().invert_yaxis()
plt.gca().invert_xaxis()
plt.title("Mapa de Ocupação - A* + DWA", fontsize=16)
plt.show()

# Restrições cinemáticas e constantes físicas do robô diferencial
R, L = 0.036, 0.235
max_v, min_v, max_w = 0.12, -0.03, 0.6
max_accel_v, max_accel_w = 0.3, 0.6
alpha, beta, gamma = 4.0, 0.8, 2.0

# Inicialização das variáveis de estado e filtros anti-tremor
v_atual, w_atual = 0.0, 0.0
w_anterior = 0.0
current_key_idx = 1
tempo_acumulado = 0.0
intervalo_print = 2.0
em_centralizacao = False
contador_centralizacao = 0
trajetoria_x, trajetoria_y = [], []

try:
    # Loop dinâmico iterativo de navegação local
    while current_key_idx < len(key_x):
        dt = sim.getSimulationTimeStep()
        tempo_acumulado += dt

```

```

pos = sim.getObjectPosition(robotBase, -1)
ori = sim.getObjectOrientation(robotBase, -1)
theta = ori[2] + math.pi

trajetoria_x.append(pos[0])
trajetoria_y.append(pos[1])

local_gx, local_gy = key_x[current_key_idx],
key_y[current_key_idx]
dist_to_local = math.hypot(local_gx - pos[0], local_gy - pos[1])

# Valida transição e avanço para próxima submeta global
if dist_to_local < 0.25:
    print(f"    ✅ Submeta {current_key_idx}/{len(key_x)-1}
alcançada!")
    current_key_idx += 1
    continue

# Valida proximidade com o objetivo absoluto final
if math.hypot(gx - pos[0], gy - pos[1]) < 0.2:
    print("\n🎯 [SUCESSO] Destino final alcançado!")
    break

# Leitura analítica do array de sensores de proximidade
distancias_obstaculos = []
for i in range(5):
    res, dist, _, _, _ = sim.readProximitySensor(sensores[i])
    distancias_obstaculos.append(dist if (res > 0 and dist < 0.5)
else 0.5)

# Avalia e categoriza a geometria livre lateral (estreitamentos)
tem_obstaculo_esquerda = distancias_obstaculos[0] < 0.25 or
distancias_obstaculos[1] < 0.25
tem_obstaculo_direita = distancias_obstaculos[3] < 0.25 or
distancias_obstaculos[4] < 0.25
tem_obstaculo_frente = distancias_obstaculos[2] < 0.25
passagem_estreita = tem_obstaculo_esquerda and
tem_obstaculo_direita and not tem_obstaculo_frente

```

```

        # Classifica barreiras locais (estáticas conhecidas vs dinâmicas
surpresas)

        obstaculo_no_mapa = False
        obstaculo_fora_mapa = False
        for i in range(5):
            dist = distancias_obstaculos[i]
            if dist < 0.5:
                angulo_sensor = get_sensor_angle(i)
                obs_x = pos[0] + dist * math.cos(theta + angulo_sensor)
                obs_y = pos[1] + dist * math.sin(theta + angulo_sensor)
                if is_obstacle_in_ogm(obs_x, obs_y, a_star, grid_size):
                    obstaculo_no_mapa = True
                else:
                    obstaculo_fora_mapa = True

        # Calcula erros de orientação baseados na pose instantânea
        angulo_alvo = math.atan2(local_gy - pos[1], local_gx - pos[0])
        erro_angular = math.atan2(math.sin(angulo_alvo - theta),
math.cos(angulo_alvo - theta))
        erro_heading = abs(erro_angular)

        # PRIORIDADE 1: Desvio de Obstáculo Frontal / Pedestres (Reação
Indispensável e Crítica)
        if distancias_obstaculos[2] < 0.25:
            espaco_esq = distancias_obstaculos[0] +
distancias_obstaculos[1]
            espaco_dir = distancias_obstaculos[3] +
distancias_obstaculos[4]
            perigo_esq = distancias_obstaculos[0] < 0.15 or
distancias_obstaculos[1] < 0.15
            perigo_dir = distancias_obstaculos[3] < 0.15 or
distancias_obstaculos[4] < 0.15

            v_atual = 0.04 if (perigo_esq or perigo_dir) else 0.05
            if perigo_esq and perigo_dir:
                w_atual = 0.4 if espaco_esq > espaco_dir else -0.4
            elif perigo_esq: w_atual = -0.5

```

```

        elif perigo_dir: w_atual = 0.5
        else: w_atual = 0.5 if espaco_esq > espaco_dir else -0.5

        if tempo_acumulado >= intervalo_print:
            print(f"[STATUS] 🔄 DESVIO (Frente) | Vel: {v_atual:.2f} | W: {w_atual:.2f}")
            tempo_acumulado = 0.0

            sim.setJointTargetVelocity(motorEsquerdo, (v_atual - (w_atual * L / 2)) / R)
            sim.setJointTargetVelocity(motorDireito, (v_atual + (w_atual * L / 2)) / R)

            w_anterior = w_atual
            client.step()
            continue

# PRIORIDADE 1.5: Centralização Suave entre Paredes (Filtro Local Ativo)

d_esq = min(distancias_obstaculos[0], distancias_obstaculos[1])
d_dir = min(distancias_obstaculos[3], distancias_obstaculos[4])
diferenca_lateral = abs(d_esq - d_dir)
tem_lateral_proximo = d_esq < 0.25 or d_dir < 0.25

if diferenca_lateral < 0.08 or contador_centralizacao > 30:
    em_centralizacao = False
    contador_centralizacao = 0

if tem_lateral_proximo and diferenca_lateral > 0.12 and not tem_obstaculo_frente and not em_centralizacao:
    em_centralizacao = True
    contador_centralizacao = 0

if em_centralizacao:
    contador_centralizacao += 1
    if diferenca_lateral < 0.08 or contador_centralizacao > 30:
        em_centralizacao, contador_centralizacao = False, 0
    else:
        erro_centro = d_dir - d_esq

```



```

w_centro = -0.4 * (erro_centro / 0.25)
w_alvo = max(-0.25, min(0.25, w_centro))

# Suaviza a variação da aceleração angular na
centralização

max_dw = 0.8 * dt
w_atual = max(w_anterior - max_dw, min(w_anterior +
max_dw, w_alvo))
w_anterior = w_atual

v_atual = min(v_atual + max_accel_v * dt, max_v * 0.8)

if tempo_acumulado >= intervalo_print:
    print(f"[STATUS] ⚖️ CENTRALIZAÇÃO | W:
{w_atual:+.2f}")
    tempo_acumulado = 0.0

sim.setJointTargetVelocity(motorEsquerdo, (v_atual -
(w_atual * L / 2)) / R)
sim.setJointTargetVelocity(motorDireito, (v_atual +
(w_atual * L / 2)) / R)
client.step()
continue

# PRIORIDADE 2: Confiança Absoluta no Planejador Global A*
(Controle Proporcional)
if obstaculo_no_mapa and not obstaculo_fora_mapa:
    if erro_heading < 0.5:
        v_atual = min(v_atual + max_accel_v * dt, max_v)
        erro_centro_p2 = d_dir - d_esq
        if abs(erro_centro_p2) > 0.08 and (d_esq < 0.25 or d_dir <
0.25):
            w_alvo = max(-0.30, min(0.30, -0.4 * (erro_centro_p2 /
0.25)))
        else:
            w_alvo = 0.0
    else:
        v_atual = 0.03

```

```

        w_alvo = max(-0.8, min(0.8, 1.5 * erro_angular))

        # Suaviza e amortece o comando de virada da rota global
        max_dw = 1.2 * dt
        w_atual = max(w_anterior - max_dw, min(w_anterior + max_dw,
w_alvo))

        w_anterior = w_atual

        if tempo_acumulado >= intervalo_print:
            print(f"[STATUS] 🚗 A* Coordena | Vel: {v_atual:.2f}")
            tempo_acumulado = 0.0

        sim.setJointTargetVelocity(motorEsquerdo, (v_atual - (w_atual
* L / 2)) / R)

        sim.setJointTargetVelocity(motorDireito, (v_atual + (w_atual *
L / 2)) / R)

        client.step()
        continue

    # PRIORIDADE 3: Alinhamento de Passagem Estreita (Filtro Local
Ativo)

    if passagem_estreita and distancias_obstaculos[2] > 0.20:
        v_atual = min(v_atual + max_accel_v * dt, max_v)
        w_alvo = max(-0.15, min(0.15, 0.8 * erro_angular))

        # Filtro passa-baixa local para o alinhamento em corredores
apertados

        max_dw = 0.8 * dt
        w_atual = max(w_anterior - max_dw, min(w_anterior + max_dw,
w_alvo))

        w_anterior = w_atual

        if tempo_acumulado >= intervalo_print:
            print(f"[STATUS] 🚶 Passagem Direta | Vel: {v_atual:.2f}")
            tempo_acumulado = 0.0

        sim.setJointTargetVelocity(motorEsquerdo, (v_atual - (w_atual
* L / 2)) / R)

```

```

        sim.setJointTargetVelocity(motorDireito, (v_atual + (w_atual *
L / 2)) / R)

        client.step()

        continue

# PRIORIDADE 4: Janela Dinâmica Tradicional DWA (Ambientes Livres
e Abertos)

v_min_dw = max(min_v, v_atual - max_accel_v * dt)
v_max_dw = min(max_v, v_atual + max_accel_v * dt)
w_min_dw = max(-max_w, w_atual - max_accel_w * dt)
w_max_dw = min(max_w, w_atual + max_accel_w * dt)

melhor_v, melhor_w, max_score = 0.0, 0.0, -9999.0
v_step = (v_max_dw - v_min_dw) / 5.0 if v_max_dw > v_min_dw else
0.1
w_step = (w_max_dw - w_min_dw) / 10.0 if w_max_dw > w_min_dw else
0.1

v = v_min_dw
while v <= v_max_dw:
    w = w_min_dw
    while w <= w_max_dw:
        theta_futuro = theta + w * 0.8
        x_futuro = pos[0] + v * math.cos(theta_futuro) * 0.8
        y_futuro = pos[1] + v * math.sin(theta_futuro) * 0.8

        angulo_alvo_pred = math.atan2(local_gy - y_futuro,
local_gx - x_futuro)
        erro_heading_pred =
abs(math.atan2(math.sin(angulo_alvo_pred - theta_futuro),
math.cos(angulo_alvo_pred - theta_futuro)))

        score_heading = 1.0 - (erro_heading_pred / math.pi)

        min_dist = min(distancias_obstaculos)
        score_clearance = 1.0

```

```

# Restrição e penalidade fina baseada na tolerância de
15cm

    if min_dist < 0.35:
        score_clearance = min_dist / 0.35
        if (distancias_obstaculos[0] < 0.15 or
distancias_obstaculos[1] < 0.15) and w > 0.2: score_clearance -= 0.3
        if (distancias_obstaculos[3] < 0.15 or
distancias_obstaculos[4] < 0.15) and w < -0.2: score_clearance -= 0.3
        if distancias_obstaculos[2] < 0.15 and v >= 0.05:
score_clearance -= 0.5

        score_vel = (v / max_v) if max_v > 0 else 0
        if erro_heading_pred < 0.3 and min_dist > 0.2:
            score_vel = min(score_vel * 1.5, 1.0)

        score = (alpha * score_heading) + (beta * score_clearance)
+ (gamma * score_vel)
        if v > 0.02: score += 0.5

        if score > max_score:
            max_score, melhor_v, melhor_w = score, v, w
            w += w_step
            v += v_step

    v_atual = melhor_v

# Filtro passa-baixa local para o comando estabilizado da janela
DWA

    max_dw = 1.2 * dt
    w_atual = max(w_anterior - max_dw, min(w_anterior + max_dw,
melhor_w))
    w_anterior = w_atual

# SALVAGUARDA ABSOLUTA: Parada e Manobra de Emergência de Último
Recurso (<10cm)
    if min(distancias_obstaculos) < 0.10:
        v_atual = -0.02
        esp_esq = distancias_obstaculos[0] + distancias_obstaculos[1]

```

```

        esp_dir = distancias_obstaculos[3] + distancias_obstaculos[4]
        w_atual = 0.6 if esp_esq > esp_dir else -0.6
        w_anterior = w_atual
        print(f"      ⚠ EMERGENCY OVERRIDE! Distância Crítica:
{min(distancias_obstaculos):.2f}m")

        # Executa envio físico final das velocidades cinemáticas
calculadas
        w_atual = max(-max_w, min(max_w, w_atual))
        sim.setJointTargetVelocity(motorEsquerdo, (v_atual - (w_atual * L
/ 2)) / R)
        sim.setJointTargetVelocity(motorDireito, (v_atual + (w_atual * L /
2)) / R)

        if tempo_acumulado >= intervalo_print:
            print(f"[STATUS] 🤖 DWA Normal | Vel: {v_atual:.3f} | W:
{w_atual:.2f}")
            tempo_acumulado = 0.0

        client.step()

except KeyboardInterrupt:
    print("\n⚠ Simulação interrompida.")
finally:
    # Força parada total e finalização segura dos motores
    sim.setJointTargetVelocity(motorEsquerdo, 0.0)
    sim.setJointTargetVelocity(motorDireito, 0.0)
    sim.stopSimulation()

    # Gera o gráfico final comparativo da trajetória executada
    if trajetoria_x and trajetoria_y:
        plt.figure(figsize=(12, 10))
        plt.plot(ox, oy, ".k", markersize=2, alpha=0.3)
        plt.plot(key_x, key_y, "m--", linewidth=2, label="Rota Planejada")
        plt.plot(trajetoria_x, trajetoria_y, "g-", linewidth=3,
label="Trajetória Real")
        plt.plot(sx, sy, "og", markersize=14)
        plt.plot(gx, gy, "Xb", markersize=16)

```

```
        for i, (cx, cy) in enumerate(zip(key_x, key_y)):
            if 0 < i < len(key_x)-1: plt.plot(cx, cy, "mo", markersize=10)
plt.legend(loc="best", fontsize=11)
plt.grid(True, linestyle=":", alpha=0.3)
plt.axis("equal")
plt.gca().invert_yaxis()
plt.gca().invert_xaxis()
plt.title("Trajetória Real - DWA OGM Limpo e Suave", fontsize=16)
plt.show()

print("\n[FIM] Simulação encerrada.")

if __name__ == '__main__':
    main()
```

6. REFERÊNCIAS

LI, Xiao; HU, Xu; WANG, Zhe; DU, Zhijun. "Path planning based on combination of improved A-STAR algorithm and DWA algorithm". In: 2nd International Conference on Artificial Intelligence and Advanced Manufacture (AIAM), 2020, Manchester, Reino Unido. IEEE, 2020.

FOX, Dieter; BURGARD, Wolfram; THRUN, Sebastian. "The dynamic window approach to collision avoidance". IEEE Robotics & Automation Magazine, v. 4, n. 1, p. 23-33, 1997.

SANTANA, Ismael Cruz de. "Modelagem e Simulação do Robô iCreate 2 no CoppeliaSim". Trabalho de Conclusão de Curso, Engenharia Mecânica, UNIVASF, Juazeiro-BA, 2025.

RODRIGUES, Danilo Ribeiro et al. "Trabalho 1 - Janela Dinâmica". Disciplina de Tópicos Avançados em Automação, Engenharia da Computação, UNIVASF, Juazeiro-BA, 2021.

SAKAI, Atsushi et al. "PythonRobotics: a Python code collection of robotics algorithms". Disponível em: <https://github.com/AtsushiSakai/PythonRobotics>.

SAKAI, Atsushi. "a_star.py": Implementação de referência do algoritmo A*, módulo PathPlanning/AStar, repositório PythonRobotics. Disponível em: https://github.com/AtsushiSakai/PythonRobotics/blob/master/PathPlanning/AStar/a_star.py.

MORAVEC, Hans; ELFES, Alberto. "High resolution maps from wide angle sonar". In: Proceedings. 1985 IEEE International Conference on Robotics and Automation. Silver Spring, MO: IEEE Computer Society Press, 1984. p. 116-121.

COPPELIA ROBOTICS. "CoppeliaSim User Manual".

IROBOT CORPORATION. "iRobot Create 2 Anatomy & Anatomy Guide".